



Faculty of Computing
Department of Computing

Laboratory Manual

CS-250 : Data Structure and Algorithms

Class: BESE 16AB • Semester: Fall 2026 – 3rd Semester

Experiment No. 02

Dynamic Memory Allocation with Git and Unit Testing

Date	18 September 2026
Time	09:00 AM – 11:50 AM & 2:00 PM – 4:50 PM
Instructor(s)	Dr Faisal Shafait
Lab Engineer	Hamza Fayyaz
Mapped CLO	CLO2 : Solve a given real time problem by applying appropriate data structure and algorithm. CLO3 : Practice programs using the latest IDEs ensuring testing, documentation and packaging of programs as per standards practices applicable to the software industry. CLO4 : Demonstrate commitment to professional ethics by following engineering norms applicable to the software industry.:

1 Introduction

This lab focuses on two key aspects of software development:

1. **Dynamic Memory Allocation in C++** – a crucial programming concept that allows programmers to allocate and deallocate memory at runtime using the heap. This enables handling variable-sized data efficiently.
2. **Git & Unit Testing** – professional software development practices. Git is an industry-standard version control system used for managing code changes and collaboration. Unit testing ensures correctness of individual program components.

Students will implement C++ programs using dynamic memory allocation while also managing their work with GitHub and writing simple unit tests.

2 Objectives

- Understand the difference between stack and heap memory in C++.
- Learn how to allocate and deallocate memory dynamically using `new` and `delete`.
- Practice version control using GitHub (`init`, `clone`, `add`, `commit`, `push`, `pull`, `branch`, `merge`).
- Learn to write basic unit tests to verify correctness of C++ functions.
- Integrate coding, testing, and version control into a complete workflow.

3 Concepts Covered

3.1 Dynamic Memory Allocation in C++

- Stack vs Heap memory.
- `new` and `delete` operators.
- Resizing arrays using dynamic allocation.
- Creating dynamic 2D arrays.

3.2 Git Basics

- `git init`, `git clone`, `git status`, `git add`, `git commit`.
- `git push`, `git pull`.
- Branching and merging (`git branch`, `git checkout -b`, `git merge`).
- GitHub repository setup and pull requests.

3.3 Unit Testing

- What is a unit test?
- Writing tests for small functions.
- Using frameworks.
- Running and interpreting test results.

4 Tools/Software Requirement

Visual Studio C++ / VS Code
Git installed locally (or GitHub Desktop)
GitHub account

5 Description: Static vs Dynamic Arrays

Consider two variants of declaring arrays below. Memory for the first variant gets allocated on the stack. The lifetime of an array created using Method A depends on its scope. If it is defined globally, its life is equal to the lifetime of the application. If it is declared in a function, memory for it gets allocated on the stack when the function gets called. It gets deallocated when the function call terminates. All the data related to the function call, including the array, gets removed from the stack.

On the other hand, memory for the array created using the new operator gets allocated on the heap at runtime. The lifetime of such an array is at maximum equal to the execution time of the application. If the array is no longer required, the memory allocated for it can be freed using `delete[]`.

Method A:

```
1  const int size = 5;
2  int x[size];
3
4  for (int i = 0; i < size; i++)
5  {
6      x[i] = i + 1;
7  }
```

Method B:

```
1  int size; // Note that size is const in Method A whereas it isn't in Method B.
2  cout << "Enter size of array: ";
3  cin >> size;
4
5  int *x = new int[size];
6
7  for (int i = 0; i < size; i++)
8  {
9      x[i] = i + 1;
10 }
```

6 Practice Tasks

6.1 Problem 1: Dynamic Array and Sum

Problem: Write a program to dynamically allocate memory for an array of integers and find the sum of all elements.

Solution Code:

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int n;
6      cout << "Enter the number of elements: ";
7      cin >> n;
8
9      // Dynamically allocate memory for an array of size n
10     int* arr = new int[n];
11
12     // Input array elements
13     cout << "Enter " << n << " elements:\n";
14     for (int i = 0; i < n; i++) {
15         cin >> arr[i];
16     }
17 }
```

```
18 // Calculate sum
19 int sum = 0;
20 for (int i = 0; i < n; i++) {
21     sum += arr[i];
22 }
23
24 cout << "Sum of array elements: " << sum << endl;
25
26 // Free dynamically allocated memory
27 delete[] arr;
28 return 0;
29 }
```

Output:

```
1 Enter the number of elements: 5
2 Enter 5 elements:
3 1 2 3 4 5
4 Sum of array elements: 15
```

6.2 Problem 2: Dynamically Resize an Array

Problem: Write a program to dynamically resize an array using `realloc` in C++ (through a workaround). Input some elements, resize the array to a larger size, and add more elements.

Solution Code:

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     int n;
6     cout << "Enter the initial number of elements: ";
7     cin >> n;
8
9     // Dynamically allocate memory for the array
10    int* arr = new int[n];
11
12    // Input initial elements
13    cout << "Enter " << n << " elements:\n";
14    for (int i = 0; i < n; i++) {
15        cin >> arr[i];
16    }
17
18    // Resize the array to a larger size
19    int newSize;
20    cout << "Enter new size (larger than " << n << "): ";
21    cin >> newSize;
22
23    int* newArr = new int[newSize];
24
25    // Copy old elements to new array
26    for (int i = 0; i < n; i++) {
27        newArr[i] = arr[i];
28    }
29
30    // Input additional elements
31    cout << "Enter additional elements:\n";
```

```
32     for (int i = n; i < newSize; i++) {
33         cin >> newArr[i];
34     }
35
36     // Output all elements
37     cout << "All elements after resizing:\n";
38     for (int i = 0; i < newSize; i++) {
39         cout << newArr[i] << " ";
40     }
41     cout << endl;
42
43     // Free dynamically allocated memory
44     delete[] arr;
45     delete[] newArr;
46     return 0;
47 }
```

Output:

```
1 Enter the initial number of elements: 3
2 Enter 3 elements:
3 1 2 3
4
5 Enter new size (larger than 3): 5
6 Enter additional elements:
7 4 5
8
9 All elements after resizing:
10 1 2 3 4 5
```

7 Lab Tasks

Task 1.

GitHub Account & Repository Setup (Step-by-Step)**I. Create a GitHub Account:**

- Go to <https://github.com>.
- Sign up with your email.
- Verify your email and set up your profile.

II. Create a New Repository:

- Click on *New Repository*.
- Repository name: DSA-Lab2.
- Choose *Public*.
- Check: *Add README.md*.
- Create the repository.

III. Clone Repository to Local Machine:

```
1 git clone https://github.com/<your-username>/DSA-Lab2.git
2 cd DSA-Lab2
```

IV. Set Git Identity:

```
1 git config --global user.name "Your Name"
2 git config --global user.email "your@email.com"
```

V. Add a Starter Program: Create hello.cpp:

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     cout << "Welcome to DSA Lab 2!" << endl;
6     return 0;
7 }
```

VI. Build and Run:

```
1 g++ hello.cpp -o hello
2 hello
```

VII. Stage, Commit, and Push:

```
1 git add hello.cpp
2 git commit -m "Add hello.cpp starter program"
3 git push origin main
```

Task 2.

Dynamic Salary Program with Git Workflow

Rewrite the following program using pointers in place of arrays (use dynamic memory allocation operators new and delete). The syntax is `int *salArray = new int[size];` and for deletion use `delete [] salArray;`.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     const int size = 20;
6     int salArray[size];
7
8     // Input salaries
9     for (int i = 0; i < size; i++) {
10         cout << "Enter Salary for employee " << (i + 1) << ": ";
11         cin >> salArray[i];
12     }
13
14     // Apply increment formula
15     for (int i = 0; i < size; i++) {
16         salArray[i] = salArray[i] + salArray[i] / (i + 1);
17     }
18
19     // Display updated salaries
20     cout << "\nUpdated Salaries: ";
21     for (int i = 0; i < size; i++) {
22         cout << salArray[i] << " ";
23     }
```

```
23     }
24
25     cout << endl;
26     return 0;
27 }
```

Steps:

1. Implement the program in `salary.cpp`.
2. Compile and test locally.
3. Use the Git workflow:

```
1 git add salary.cpp
2 git commit -m "Implement salary program with dynamic allocation"
3 git push origin main
```

Task 3.

Analyze Pointer Program with Git Workflow

Complete the following two parts for the analyze pointer problem.

Part 1: Write a function `void analyze_pointer(int *ptr)` that does two things:

- Write the memory location pointed to by the pointer to the console.
- Write the value of the integer (which the pointer points to) to the console.

Part 2: Use the function to complete two tasks:

- Allocate an `int` on the stack (e.g., `int iValue;`), assign a value to it, and get its memory location (with the reference operator `&`) to pass this value to `analyze_pointer`.
- Allocate an `int` on the heap (with the `new` operator). Assign a value to it and pass it to `analyze_pointer`.

You must upload this task to Git as well.

Task 4.

Dynamic String Reversal with Git Workflow

Write a program that dynamically allocates memory for a string, takes input from the user, reverses the string, displays the reversed result, and finally frees the allocated memory. You must also upload this program (`string_reverse.cpp`) to your GitHub repository.

Task 5.

Test-Driven Development (TDD): Checking a Sorted Array

In this task, you will practice **Test-Driven Development (TDD)** by developing a function that determines whether an integer array is sorted. You must write the test cases **before implementing the complete function**.

Implement the following function:

```
1 bool isSorted(const int* arr, const int size);
```

The function should return `true` if the elements of the array are arranged in **non-decreasing (ascending) order**. Otherwise, it should return `false`.

For example:

- {1, 2, 3, 4, 5} → true
- {1, 2, 2, 3, 5} → true
- {1, 4, 3, 5, 6} → false

Part 1 – Write the Tests First

Before implementing the complete `isSorted()` function, write test cases using `assert()` to verify its expected behavior.

At minimum, your test cases should cover:

1. An array already sorted in ascending order.
2. An unsorted array.
3. A sorted array containing duplicate values.
4. An array containing only one element.
5. An array sorted in descending order.
6. An array containing negative values.
7. **At least two additional test cases designed by you.**

Example test:

```
1 void testSortedArray() {  
2     int arr[] = {1, 2, 3, 4, 5};  
3  
4     bool result = isSorted(arr, 5);  
5  
6     assert(result == true);  
7 }
```

Students should create separate test functions, for example:

```
1 testSortedArray();  
2 testUnsortedArray();  
3 testDuplicateValues();  
4 testSingleElement();  
5 testDescendingArray();  
6 testNegativeValues();
```

Part 2 – Implement the Function

After writing the initial test cases, implement the `isSorted()` function.

Your implementation should compare neighboring elements of the array and determine whether any element appears before a smaller element.

Run the test cases after each significant change to your implementation. Continue modifying the function until **all test cases pass**.

Do not modify a correct test simply because your implementation fails it. Determine why the implementation failed and modify the implementation accordingly.

Part 3 – Demonstrate the TDD Cycle

Choose at least **one test case that initially fails** and document the TDD process in your lab report.

Include:

- The test case you wrote.
- The initial result of the test (**Pass/Fail**).
- Why the test failed.
- The modification you made to `isSorted()`.
- The result after modifying the implementation (**Pass/Fail**).

Your work should demonstrate the following TDD cycle:

Write Test → Run Test → Test Fails → Modify Code → Run Test Again → Test Passes

Part 4 – Git Workflow

Use Git to demonstrate that the test cases were written before the final implementation. Your Git commit history should clearly show the progression of your work.

Suggested commits:

```
1 Add initial tests for isSorted function
2 Implement isSorted function
3 Add edge case tests for isSorted
4 Handle additional edge cases
```

Push the completed implementation and all test cases to your **Lab 2 GitHub repository**.

Expected Outcome

At the end of this task, you should be able to:

- Write test cases before implementing a function.
- Identify normal, boundary, and edge cases.
- Use `assert()` to verify expected program behavior.
- Modify an implementation based on failing tests.
- Apply the **Test-Driven Development cycle** to a simple programming problem.
- Use Git commits to document the development and testing process.

8 Deliverables

Compile a single Word document by filling in the solution part and submit this Word file on LMS. The name of the Word document should follow this format, i.e., `YourFullName(reg)_Lab#`. You must show the implementation of the tasks in the design tool, along with your complete Word document to get your work graded. You must also submit this Word document on the LMS and provide the GitHub repository link for your lab code.

Note: Students are required to upload the lab to LMS before the deadline.

9 Lab Rubrics

Assessment	Doesn't meet Expectation (1–2)	Meets Expectation (3–4)	Exceeds Expectation (5)	Marks
Software Problem Realization (CLO2)	The Student is unable to understand and outline the problem and doesn't use the relevant method to solve it.	The student requires some guidance to completely comprehend the problem and to differentiate the data structure and algorithm comprehensively.	The student fully understands the given problem, is able to analyze the relevant method to solve it, and develops a detailed program flow.	
Software Tool Usage (CLO3)	The student has no idea on how to use the basic tools of the software. The codes have syntax errors, and parts of the codes are missing. Also, they are unable to imitate the required output.	The student has a limited command on the basic tools of the software and operated it under supervision. The codes are correct in terms of their syntax; however, the program output is not always correct.	The student has full command on various tools available in the software. Furthermore, his/her coding is complete and functional, and the program output is correct. Moreover, they can easily manipulate the code to design a particular solution.	
Ethics and Adherence to Laboratory Safety Rules (CLO4)	The student does not behave according to professional ethics by following ethical norms applicable to the software industry, such as acknowledgement while using publicly available data/code. Disturbs the lab environment, doesn't take care of safety measures, and/or isn't punctual.	The student partially demonstrates commitment to professional ethics by following ethical norms applicable to the software industry, such as referencing and acknowledgement while using publicly available data/code. Exhibits better behavior, works by taking into account safety measures, and is punctual.	The student clearly expresses commitment to professional ethics by following ethical norms applicable to the software industry, such as referencing and acknowledgement while using publicly available data/code. Encourages others to maintain lab decorum and alerts them to follow safety measures.	